

Team 9:

Cross-Continental Eco-Economic Data Pipeline

David Denice

Devothama Narasimhamurthy

Robert Hula

Natalya Ratra

EN 685.652.81.SP26 - Data Engineering Principles and Practice

April 14, 2026

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Project Overview	4
2.1 - Scope	4
3. System Deployment and Installation Guide.....	4
3.1 - Technical Setup and Execution	4
3.2 - Troubleshooting	5
3.3 - Automated Maintenance	5
3.4 - Next Steps	5
4. Data Acquisition and Integration	5
4.1 - Global Weather Data	5
4.2 - Global Food Prices	6
4.3 - International Energy Data	6
4.4 - World Bank Macroeconomic Indicators	6
5. Design Decisions and Rationale.....	6
6. Data Quality and Handling.....	7
7. Database Design.....	7
7.1 - Database Normalization and Integrity.....	7
7.2 - Data Dictionary	7
8. ETL Pipeline Architecture	8
8.1 - System Orchestration and Modules.....	8
9. System Automation and Orchestration	9
9.1 - Initialization and Historical Backfill	9
9.2 - Scheduled Maintenance and Incremental Refresh	9
10. API Documentation.....	9
10.1 - Application Interface	10
10.2 - API Endpoint Reference.....	10
10.3 - Integrated Analytical Output	10
11. Docker Containerization and Deployment.....	10
11.1 - Service Architecture	10
11.2 - Environment Configuration.....	11
11.3 - Orchestration and Lifecycle	11
12. Usage Instructions	11
12.1 - Manual ETL Execution	11

12.2 - API Consumption and Endpoint Reference	12
12.3 - System Monitoring and Troubleshooting.....	12
12.4 - Direct Database Inspection (CLI)	12
12.5 - System Lifecycle and Automation	13
13. Project Structure	13
14. Future Enhancements	14
15. Appendix	14
15.1 Sample Records	14
15.2 Example API Responses.....	17
15.3 EIA API Registration.....	19
15.4 Docker Response	20
15.5 Troubleshooting Notes	21
15.6 References	22

1. Executive Summary

This project delivers an end-to-end data engineering pipeline integrating heterogeneous international datasets into a centralized PostgreSQL database for cross-country environmental and economic analysis. By unifying weather, macroeconomic, energy, and food price data into a standardized monthly structure, the platform creates a foundation for examining how national conditions evolve, and whether shifts in one domain coincide with changes in another.

In addition to the integrated database, the project includes automated ETL orchestration, a Flask API, and a Dockerized deployment workflow. The result is a reusable framework that transforms fragmented sources into a cohesive model for comparative analysis and reporting.

2. Project Overview

2.1 - Scope

Geographic coverage includes the United States, Brazil, India, the Philippines, and Nigeria, although not every dataset covers every country. The United States and Brazil provide broader macroeconomic and energy coverage, while India, the Philippines, and Nigeria align with available food price data and introduce variation in climate exposure. The analysis begins in 2022 and extends through 2026 to ensure the overlap is analytically useful and current. The pipeline is structured to support automated weekly refreshes as new data becomes available.

3. System Deployment and Installation Guide

3.1 - Technical Setup and Execution

These instructions assume Docker and Docker Compose are already installed on the host system. This project must be executed from its own root directory (T9_final_project) and is not compatible with the pre-configured jhu_docker environment.

1. Open the project folder and ensure all core files are present (docker-compose.yml, Dockerfile, .env.example, and ETL scripts)
2. Obtain an EIA API key by registering at the [official EIA registration page](#). The key will be delivered via email
3. Create the local .env file from the provided template by running the following command in your terminal: `cp env.example .env`
4. Open .env in a text editor and enter your EIA API key in the EIA_API_KEY field. The database settings and WEATHER_BASE_URL should remain unchanged. Save the file in the project root
5. Start the full project from the project folder by running: `docker compose up --build`
Note: This builds the containers, starts PostgreSQL, launches the ETL application, initiates cron automation, and starts the Flask and Jupyter services

6. Wait for the initial startup to complete. On the first run, PostgreSQL must initialize and the full historical backfill must finish before the API becomes available; this may take several minutes
7. After startup is complete, the following interfaces are available:
 - **Flask API Dashboard:** <http://localhost:8001>
 - **API Health Check:** <http://localhost:8001/api/health>
 - **JupyterLab:** <http://localhost:8888>

3.2 - Troubleshooting

If the system fails to launch, perform the following checks:

- Verify that the .env file exists in the root directory
- Confirm that the EIA_API_KEY contains a valid, active key
- Ensure the Docker Desktop (or engine) is running
- Confirm the command was executed from the correct T9_final_project folder
- Review the real-time logs from docker compose up --build for specific error messages

3.3 - Automated Maintenance

After the first build is complete, the project continues to refresh through cron jobs.

- **Schedule:** The schedule runs /app/run_incremental.sh at 2:00 a.m. every Sunday
- **Logging:** Output is written to /app/cron.log for auditing and verification
- **Automation:** While the initial setup is manual, all future refreshes happen automatically inside the running container

3.4 - Next Steps

Once the services are verified as active, please refer to **Section 12 (Usage Instructions)** for detailed guidance on querying the API endpoints and utilizing the JupyterLab environment for data exploration.

4. Data Acquisition and Integration

All datasets are standardized around two common join keys: year_month in YYYY-MM format and ISO3 country_code values; this makes it possible to integrate sources that originally arrive in different structures. By enforcing these standardization rules across all ingest pipelines, the project ensures that disparate environmental and economic indicators can be queried as a single, cohesive dataset.

4.1 - Global Weather Data

Weather data is collected from the Open-Meteo API and aggregated from daily observations into monthly values. To ensure consistency, representative cities were selected for each country: Baltimore (USA), São Paulo (Brazil), Kolkata (India), Manila (Philippines), and Lagos (Nigeria). The resulting dataset includes average temperature (Celsius) and total monthly precipitation (mm) for each selected city.

Source: [Historical Weather API](#)

4.2 - Global Food Prices

Sourced from the World Food Programme, this static dataset contains historical monthly prices for maize, rice, and wheat flour in India, the Philippines, and Nigeria. The raw data was filtered, standardized into USD per kilogram to allow for cross-country comparison, and aggregated into national monthly averages before being pivoted into a wide format for integration.

Source: [Global - Food Prices](#)

4.3 - International Energy Data

This API source provides annual national-level metrics from the U.S. Energy Information Administration for all five target countries. To align with the project's monthly granularity, annual values were expanded via uniform interpolation. The dataset captures total electricity production and consumption, net generation, and the production/consumption of petroleum and natural gas.

Source: [U.S. Energy Information Administration](#)

4.4 - World Bank Macroeconomic Indicators

Macroeconomic indicators including GDP, inflation, and population are retrieved from the World Bank Open Data API. To maintain consistency across the integrated model, these annual values were expanded into monthly records using uniform interpolation, allowing the economic context to be analyzed alongside the monthly weather, food price, and energy datasets.

Source: [World Bank Indicators](#)

5. Design Decisions and Rationale

This project intentionally preserves asymmetrical coverage rather than hiding it. While food prices are limited to India, the Philippines, and Nigeria, macroeconomic and weather data cover all five target countries. To manage these discrepancies and varying date ranges, the following technical choices were made to ensure the integrity and interoperability of the integrated data model:

- **Standardization (ISO3 & YYYY-MM):** These keys were selected to resolve naming discrepancies across APIs and establish a consistent monthly time scale, enabling reliable cross-source joins.
- **Relational Logic (Outer Joins):** Chosen specifically to preserve data visibility. By using outer joins, the pipeline retains all available records while keeping gaps in source coverage (such as US food prices) transparently represented as null values.
- **Data Alignment (Interpolation):** To bridge the gap between annual and monthly sources, the pipeline uses uniform interpolation. This distributes annual values across months, allowing the energy and macroeconomic data to be analyzed alongside higher-frequency weather and food metrics without introducing seasonal bias.

6. Data Quality and Handling

Data integrity was maintained through automated consistency checks during the ETL process. To ensure quality, the pipeline enforces:

- **Schema Enforcement:** Every dataset is validated against a common country-by-month structure prior to the PostgreSQL load step to prevent structural mismatches.
- **Deduplication:** Aggregation and pivoting logic are applied during transformation to eliminate duplicate records and ensure a "one-row-per-country-month" grain.
- **Null Integrity:** The pipeline distinguishes between "Processing Failures" and "Source Gaps" by intentionally preserving nulls for missing coverage, ensuring the final master_data table accurately reflects original source availability.

7. Database Design

The database is implemented in PostgreSQL using a normalized relational structure. The schema is designed to enforce referential integrity across disparate sources while supporting efficient analytical queries.

7.1 - Database Normalization and Integrity

The schema is architected to prioritize data integrity and modularity through Third Normal Form (3NF):

- **Normalization Standard:** All base tables (excluding the analytical master_data table) adhere to 3NF; this ensures that every non-key attribute is functionally dependent solely on the primary keys (country_code and year-month), eliminating transitive dependencies and data redundancy.
- **Pipeline Efficiency:** Maintaining 3NF across the source tables allows for modular updates and ensures that joins remain predictable. This design makes it possible to rapidly access or scale specific datasets without risking anomalies during the ingestion process.
- **Master Data Layer:** The master_data table serves as the "denormalized" analytical layer, purposely flattened to reduce join complexity for downstream reporting and visualization.

7.2 - Data Dictionary

Table	Purpose	Primary Key	Foreign Key(s)	Columns
countries	Central dimension table for country identifiers	country_code	N/A	country_name (text)
weather	Stores rainfall and temperature data for each country	country_code, year_month	country_code	avg_temperature (numeric), total_precipitation (numeric)

energy	Stores production and consumption of different energy types by month	country_code, year_month	country_code	All numeric - electricity_consumption, electricity_production, natural_gas_consumption, natural_gas_production, petroleum_consumption, petroleum_production
food_prices	Stores the price of wheat, maize, and rice for each country during a month	country_code, year_month	country_code	All numeric - maize_price_usd_per_kg, rice_price_usd_per_kg, wheat_price_usd_per_kg
worldbank	Stores the GPD, population, and inflation rate of a country during a month	country_code, year_month	country_code	All numeric - gdp, Inflation, population
master_data	Unified table where all project data is easily retrievable for analysis	country_code, year_month	N/A	Consolidated attributes from all tables (except for country_name)

8. ETL Pipeline Architecture

The ETL pipeline is orchestrated by a central automation script, master.py, which coordinates source-specific extraction, transformation, and loading. The architecture separates the historical backfill workflow from the recurring update logic, ensuring the system is maintainable and scalable.

8.1 - System Orchestration and Modules

File	Primary Function
world_bank.py	Retrieves World Bank API indicators and applies monthly interpolation
food_backfill.py	Constructs the baseline historical price dataset prior to incremental updates
food_download.py	Automates the retrieval of raw WFP datasets for the most recent periods
food_prices.py	Merges backfill and new downloads into a unified, standardized food dataset

eia_energy.py	Extracts EIA metrics and expands annual values into the monthly target grain
weather.py	Collects city-level observations and aggregates daily readings into monthly averages.
config.py	Stores DB credentials, API keys, and shared file paths to ensure environment parity
master.py	Runs source scripts, loads warehouse tables, and executes the final master_data join logic

9. System Automation and Orchestration

The automation layer is designed so the full ETL pipeline can be executed through a single command, removing the need for manual script management.

9.1 - Initialization and Historical Backfill

The startup sequence is handled by `start_with_cron.sh`, which triggers automatically when the Docker container launches via `docker-compose up --build`. This script ensures that the data warehouse is fully constructed before the application begins serving requests by following these steps:

- **Database Readiness:** The script waits for PostgreSQL to become available to ensure stable connectivity.
- **Master Execution:** It runs `master.py` with `PIPELINE_MODE=backfill` to execute the full historical ETL workflow.
- **System Provisioning:** It installs the **cron** configuration, writes the required environment variables to **cron.env**, and starts the cron service.
- **API Launch:** Launches the Flask API (`app.py`) only after the warehouse build is confirmed.

9.2 - Scheduled Maintenance and Incremental Refresh

To keep the data current without rerunning the full historical backfill, the project utilizes a recurring refresh workflow:

- **Orchestration Logic:** The main logic is implemented in `master.py`, which coordinates source-specific modules and refreshes the warehouse tables. In incremental mode, only newly available data is processed.
- **Scheduled Execution:** Handled by `run_incremental.sh` via a cron job inside the container.
- **Timing and Logging:** The schedule is set for 2:00 a.m. every Sunday. Output is directed to `cron.log` for troubleshooting, creating an automated weekly refresh cycle.

10. API Documentation

The project includes a Flask API implemented in `app.py` that serves as the access layer for the PostgreSQL warehouse. This API exposes processed data through JSON endpoints, allowing users to perform cross-source analysis without direct SQL querying.

10.1 - Application Interface

- **Host/Port:** The application is containerized and configured to serve requests on **Port 8001**.
- **Protocol:** All endpoints utilize the **GET** method and return standardized JSON payloads.
- **Functionality:** app.py manages the database connection pool, executes optimized queries against the warehouse, and handles the serialization of records into JSON format.

10.2 - API Endpoint Reference

The following endpoints provide access to the individual fact tables and the final integrated data model:

Endpoint	Data Source	Key Metrics Returned
/api/health	System	API and Database connection status
/api/get_countries	countries	Country codes and full names
/api/get_world_bank	world_bank	GDP, Inflation, Population
/api/get_food_prices	food_prices	Maize, Rice and Wheat Flour prices
/api/get_energy	energy	Electricity, Natural Gas, and Petroleum (Prod/Cons)
/api/get_weather	weather	Average Temperature and Total Precipitation.
/api/get_all	master_data	Unified country-by-month analytical view.

10.3 - Integrated Analytical Output

The primary output of the project is delivered via the /api/get_all endpoint. Rather than requiring users to merge multiple datasets, this route pulls directly from the master_data table. It returns a combined response containing the full 14-variable set (Economic, Food, Energy, and Weather). This serves as the project's main analytical engine, enabling cross-source correlation in a single, efficient API call.

11. Docker Containerization and Deployment

The project is containerized so the full system can run in a clean and reproducible environment using Docker and Docker Compose. This keeps the database, ETL pipeline, automation workflow, API, and notebook environment in a consistent runtime environment across systems. This setup allows the warehouse to be built, refreshed on a schedule, exposed through the API, and explored through Jupyter from the same application stack.

11.1 - Service Architecture

The deployment is managed through three specialized services to ensure a clean separation of concerns:

Service	Component	Host Port	Key Function
db	PostgreSQL 15	5432	Stores project data in a persistent volume, so the database survives container restarts
app	Project Image	8001	The main runtime for the ETL pipeline, cron automation, and Flask API
jupyter	Project Image	8888	Provides a JupyterLab environment for interactive testing and data exploration

11.2 - Environment Configuration

Connectivity and API access are managed through a centralized .env file. This allows for secure, runtime configuration of:

- **Database Settings:** DB_HOST, DB_PORT, DB_NAME, DB_USER, and DB_PASSWORD.
- **External API Access:** EIA_API_KEY and WEATHER_BASE_URL for source connectivity.
- **Internal Networking:** API_BASE_URL is used by the Jupyter service to communicate with the Flask API within the Docker network.
- **Setup Template:** An .env.example file is provided as a template for these values.

11.3 - Orchestration and Lifecycle

The Dockerfile builds the application image by installing dependencies and copying project files. The docker-compose.yml then orchestrates the service lifecycle:

- **Dependency Chain:** The app and jupyter services depend on the database health check to ensure PostgreSQL is ready before they launch.
- **Startup Sequence:** Upon execution, start_with_cron.sh waits for the database, runs the initial ETL backfill, and installs the cron schedule before launching the API.
- **Persistence:** A persistent Docker volume is mapped to the db service, ensuring the warehouse remains intact across container lifecycles.

12. Usage Instructions

Once the system is deployed and the historical backfill is complete, the following methods can be used to manage the pipeline, query the API, and inspect the underlying database.

12.1 - Manual ETL Execution

While the system is automated, users can manually trigger the pipeline for testing or ad-hoc updates using the following commands:

- **Full Pipeline Rebuild:** Reruns the entire ETL workflow and refreshes all warehouse tables
 - `docker compose exec app python master.py`
- **Targeted Module Updates:** To update a specific dataset without rebuilding the entire warehouse, execute individual source scripts:
 - `docker compose exec app python world_bank.py`
 - `docker compose exec app python eia_energy.py`
 - `docker compose exec app python food_prices.py`
 - `docker compose exec app python weather.py`

12.2 - API Consumption and Endpoint Reference

The Flask API serves as the primary interface for retrieving warehouse data. To access the API, open a browser and use the following routes:

- **Application Homepage:** <http://localhost:8001>
- **System Health Check:** <http://localhost:8001/api/health>
 - A successful response confirms the API is active and the database connection is available
- **Integrated Analytical View:** http://localhost:8001/api/get_all
 - This is the main analytical endpoint; it returns combined data from the warehouse in a single response
- **Individual Fact Tables:**
 - http://localhost:8001/api/get_countries
 - http://localhost:8001/api/get_world_bank
 - http://localhost:8001/api/get_food_prices
 - http://localhost:8001/api/get_energy
 - http://localhost:8001/api/get_weather

12.3 - System Monitoring and Troubleshooting

To verify service status or review execution logs, use the following Docker commands:

- **Check Service Status:** `docker compose ps`
- **View Historical Logs:** `docker compose logs`
- **Follow Real-time Logs:** `docker compose logs -f app`

12.4 - Direct Database Inspection (CLI)

To inspect the PostgreSQL database directly, enter the database container shell:

- `docker compose exec db psql -U jhu -d t9pipe`

Once inside the PostgreSQL environment, you can utilize the following commands:

- **List Tables:** `\dt`
- **Query Master Data:** `SELECT * FROM master_data LIMIT 10;`

12.5 - System Lifecycle and Automation

- **Automated Refreshes:** After the initial build, the project supports automated refreshes through cron. The schedule executes `/app/run_incremental.sh` at 2:00 a.m. every Sunday, handling later source updates automatically
- **Stopping the Project:** `docker compose down`
- **Restarting/Rebuilding:** `docker compose up --build`

13. Project Structure

The following directory tree outlines the organization of the project's core components, automation scripts, and data storage.

1. Root directory
 - **docker-compose.yml:** Orchestrates the db, app, and jupyter services
 - **Dockerfile:** Defines the environment and dependencies for the application image
 - **.env.example:** Template for environment variables and API keys
 - **requirements.txt:** Python library dependencies
 - **README.md:** High-level project documentation
2. Core Orchestration and Infrastructure
 - **master.py:** Central ETL orchestrator for historical and incremental runs
 - **config.py:** Shared configuration and database connection parameters
 - **app.py:** Flask API implementation and endpoint definitions
3. ETL Source Modules
 - **world_bank.py:** World Bank API extraction and transformation
 - **eia_energy.py:** EIA API extraction and transformation
 - **food_download.py:** WFP raw data ingestion
 - **food_backfill.py:** Historical food price baseline construction
 - **food_prices.py:** Food price data integration and processing
 - **weather.py:** Weather data collection and monthly aggregation
4. Shell Scripts & Automation
 - **start_with_cron.sh:** Primary container startup and backfill script
 - **run_incremental.sh:** Script triggered by cron for weekly refreshes
 - **start.sh:** Secondary/Legacy startup script
 - **cronjob:** Cron schedule configuration file
5. Testing & Documentation
 - **api_smoke_test.py:** Automated validation for API endpoints
 - **api_check.ipynb:** Interactive notebook for data exploration and verification

- **docs/**: Directory containing the ERD image and supporting technical documentation
6. Data Storage (Persistent Volumes)
- **data/raw/**: Storage for unmodified data retrieved from external sources
 - **data/processed/**: Staging area for cleaned and standardized datasets

14. Future Enhancements

Future enhancements will focus on expanding the historical depth, geographical coverage, and analytical range of the pipeline. A natural next step involves backfilling additional years of historical data prior to 2022 to support long-term trend analysis and a stronger cross-country context. The dataset could be further improved by incorporating more countries and expanding the variety of weather, macroeconomic, and energy indicators within the integrated model.

From an engineering perspective, the project could be strengthened through further normalization, specifically the implementation of a `date_dim` dimension table to improve query performance and support complex temporal filtering. Additionally, transitioning from a static reporting style to a dynamic dashboard would allow for real-time data exploration and visualization directly from the API. These improvements would transform the pipeline into a highly scalable, long-term analytical platform.

15. Appendix

15.1 Sample Records

This subsection includes representative rows from the transformed source outputs and the integrated warehouse tables. These samples demonstrate the final structure of the cleaned data and confirm that the datasets were standardized successfully before loading into the production schema.

15.1.1 - weather table

Query: Query History

```

1 --Weather table sample
2 SELECT * FROM weather LIMIT 5;
3
4

```

Data Output Messages Notifications

	country_code [PK] text	year_month [PK] text	avg_temperature double precision	total_precipitation double precision
1	BRA	2022-01	21.72903225806452	290.6
2	BRA	2022-02	21.410714285714285	165.6
3	BRA	2022-03	22.19677419354839	169.6
4	BRA	2022-04	19.56	62.4
5	BRA	2022-05	16.506451612903227	24.4

Showing rows: 1 to 5

15.1.2 - food_prices table

Query: Query History

```

1 --Food prices table sample
2 SELECT * FROM food_prices LIMIT 5;
3
4

```

Data Output Messages Notifications

	country_code [PK] text	year_month [PK] text	rice_price_usd_per_kg double precision	wheat_fLOUR_price_usd_per_kg double precision
1	IND	2023-01	0.4747474747474747	0.4661538461538461
2	IND	2023-02	0.467710843272494	0.467821223012048
3	IND	2023-03	0.4699090909090909	0.45182692076923
4	IND	2023-04	0.4782976723404255	0.4647672340425532
5	IND	2023-05	0.4715043478260869	0.4584761904761905

Showing rows: 1 to 5

15.1.3 - energy table

The screenshot shows a database client interface with a query window containing the following SQL query:

```
--energy table sample
SELECT * FROM energy LIMIT 5;
```

The results pane displays the following data:

	country_code [PK] text	year_month [PK] text	electricity_consumption double precision	electricity_production double precision	natural_gas_consumption double precision	natural_gas_production double precision	petroleum_consumption double precision	petroleum_production double precision
1	BRA	2022-01	2299.955	2544.7348	91.5866	65.6108	478.3513	6.0
2	BRA	2022-02	2299.955	2544.7348	91.5866	65.6108	478.3513	6.0
3	BRA	2022-03	2299.955	2544.7348	91.5866	65.6108	478.3513	6.0
4	BRA	2022-04	2299.955	2544.7348	91.5866	65.6108	478.3513	6.0
5	BRA	2022-05	2299.955	2544.7348	91.5866	65.6108	478.3513	6.0

15.1.4 - worldbank table

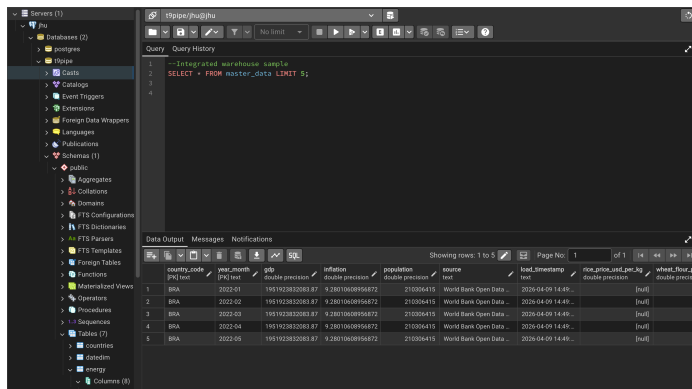
The screenshot shows a database client interface with a query window containing the following SQL query:

```
--World Bank table sample
SELECT * FROM worldbank LIMIT 5;
```

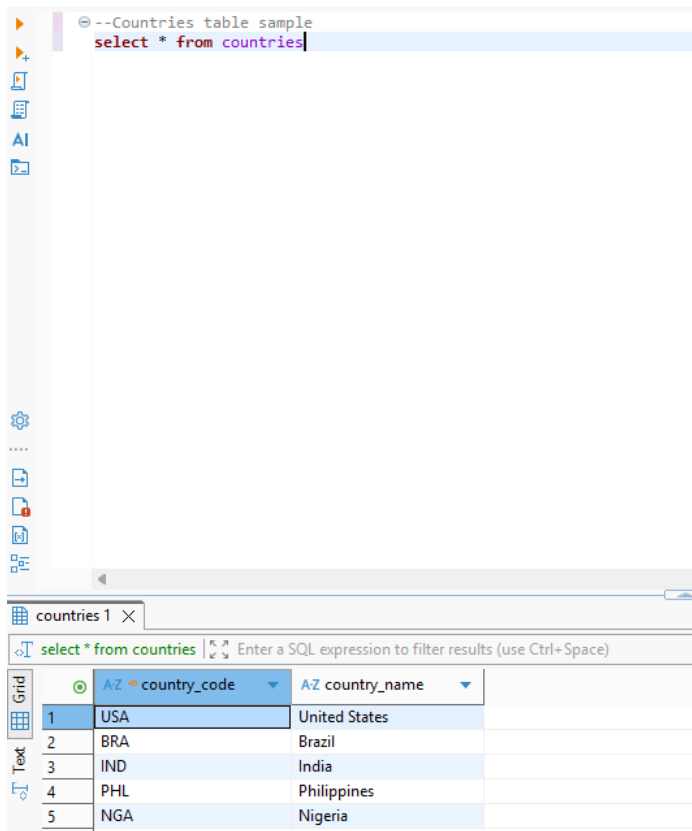
The results pane displays the following data:

	country_code [PK] text	year_month [PK] text	gdp double precision	inflation double precision	population double precision	source text	load_timestamp text
1	BRA	2022-01	1951923832083.87	9.28010608956872	210306415	World Bank Open Data ...	2026-04-09 14:49...
2	BRA	2022-02	1951923832083.87	9.28010608956872	210306415	World Bank Open Data ...	2026-04-09 14:49...
3	BRA	2022-03	1951923832083.87	9.28010608956872	210306415	World Bank Open Data ...	2026-04-09 14:49...
4	BRA	2022-04	1951923832083.87	9.28010608956872	210306415	World Bank Open Data ...	2026-04-09 14:49...
5	BRA	2022-05	1951923832083.87	9.28010608956872	210306415	World Bank Open Data ...	2026-04-09 14:49...

15.1.5 - master_data table



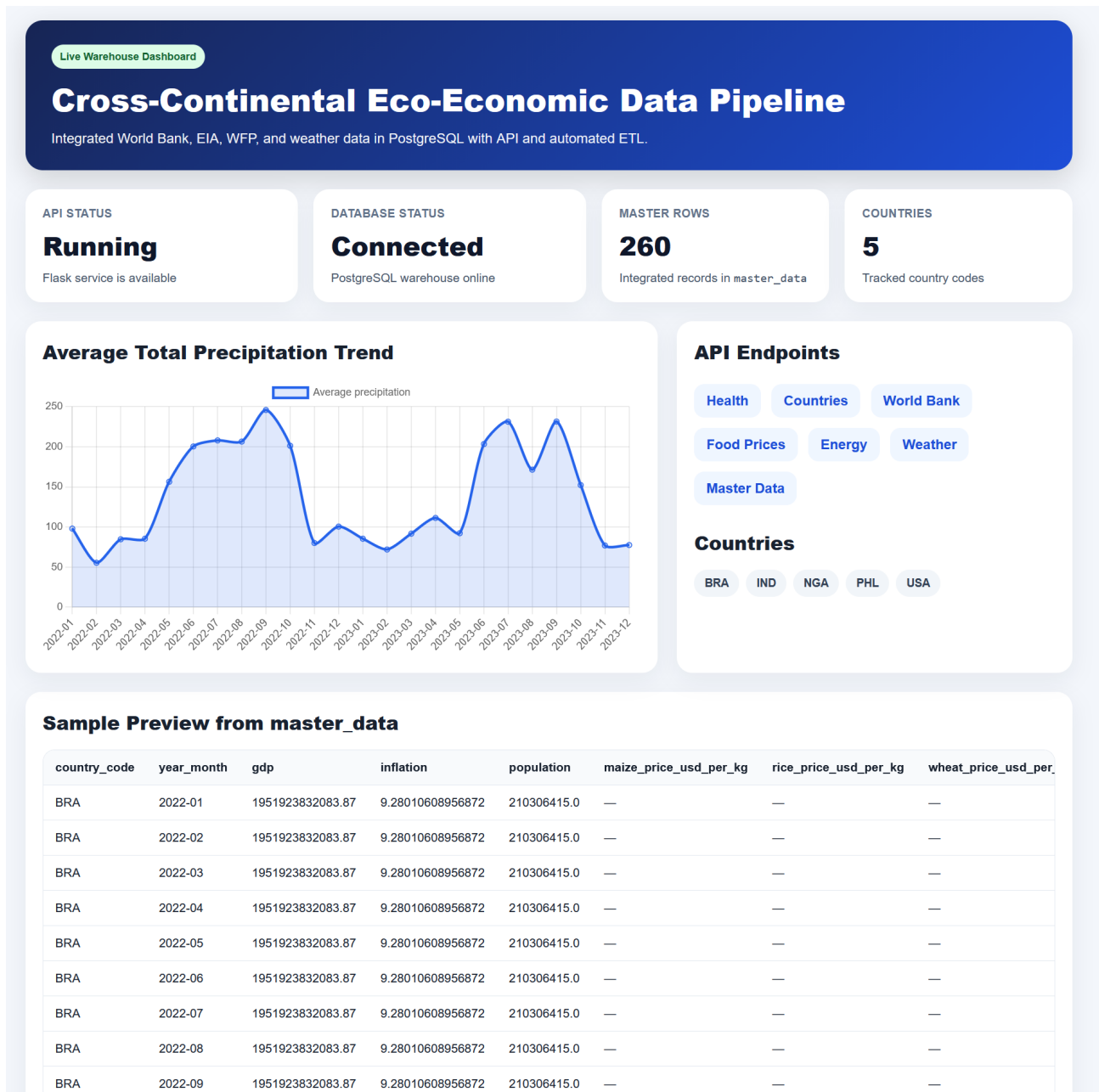
15.1.6 - countries table



15.2 Example API Responses

This subsection provides example JSON responses from the Flask API. These examples illustrate the structure and metadata provided to downstream analytical tools or front-end applications.

15.2.1 - Response from <http://localhost:8001>



15.2.2 - Response from <http://localhost:8001/api/health>

```
{
  "check": 1,
  "database": "connected",
  "status": "ok"
}
```

15.2.3 response from http://localhost:8001/api/get_all

```
[
  {
    "avg_temperature": 21.7290322580645,
    "country_code": "BRA",
    "electricity_consumption": 2299.555,
    "electricity_production": 2544.7348,
    "gdp": 1951923832083.87,
    "inflation": 9.28010608956872,
    "natural_gas_consumption": 91.5866,
    "natural_gas_production": 65.6108,
    "petroleum_consumption": 478.3513,
    "petroleum_production": 6.5257,
    "population": 210306415,
    "rice_price_usd_per_kg": null,
    "total_precipitation": 290.6,
    "wheat_flour_price_usd_per_kg": null,
    "year_month": "2022-01"
  },
  {
    "avg_temperature": 21.4107142857143,
    "country_code": "BRA",
    "electricity_consumption": 2299.555,
    "electricity_production": 2544.7348,
    "gdp": 1951923832083.87,
    "inflation": 9.28010608956872,
    "natural_gas_consumption": 91.5866,
    "natural_gas_production": 65.6108,
    "petroleum_consumption": 478.3513,
    "petroleum_production": 6.5257,
    "population": 210306415,
    "rice_price_usd_per_kg": null,
    "total_precipitation": 105.6,
    "wheat_flour_price_usd_per_kg": null,
    "year_month": "2022-02"
  },
]
```

15.3 EIA API Registration

Register for an API key

Users of the EIA API are required to obtain an key. Your API key will be sent to your email address as well as any required API notifications.

An API key is not required to use the [bulk download facility](#).

REGISTER

[FORGOT MY API KEY](#)

*First Name:

*Last Name:

*Email:

*Category that best describes you/your organization:

Business/Industry



Reason to use this API:

15.4 Docker Response

These series of screenshots help verify that the Docker environment, ETL pipeline, API, and database are running successfully from the “docker compose up –build” command.

```
(base) huerequeque@HuerequequesMBP Automated_Pipeline_Submission % docker compose up --build
[+] Building 4.1s (14/14) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 596B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 342B 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 0.6s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/7] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d12 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d12 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 11.44kB 0.0s
=> CACHED [2/7] WORKDIR /app 0.0s
=> CACHED [3/7] RUN apt-get update && apt-get install -y cron && rm -rf 0.0s
=> CACHED [4/7] COPY requirements.txt . 0.0s
=> CACHED [5/7] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [6/7] COPY . . 0.2s
=> [7/7] RUN chmod +x start.sh run_incremental.sh start_with_cron.sh 0.2s
=> exporting to image 2.4s
=> => exporting layers 1.6s
=> => exporting manifest sha256:59a4c152ede09197dadd2b8beaa253e2f63bf141 0.0s
=> => exporting config sha256:62bea4173ab812c7aa7172d5d5de9afa32aac55d85 0.0s
=> => exporting attestation manifest sha256:4b3275dac1ca83231047f9b4031b 0.0s
=> => exporting manifest list sha256:389d4cd5e89920c3eb2b1fb9b4dbf3dade8 0.0s
=> => naming to docker.io/library/automated_pipeline_submission-app:late 0.0s
=> => unpacking to docker.io/library/automated_pipeline_submission-app:late 0.7s
=> resolving provenance for metadata file 0.0s
[+] up 4/4
✔ Image automated_pipeline_submission-app Built 4.3s
✔ Network automated_pipeline_submission_default Created 0.1s
✔ Container automated_pipeline_submission-db-1 Created 0.3s
✔ Container automated_pipeline_submission-app-1 Created 0.1s
```

```

Attaching to app-1, db-1
db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-1 |
db-1 | 2026-04-10 00:38:12.841 UTC [1] LOG:  starting PostgreSQL 15.17 (Debian 15.17-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-10) 14.2.0, 64-bit
db-1 | 2026-04-10 00:38:12.841 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-04-10 00:38:12.841 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
db-1 | 2026-04-10 00:38:12.844 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-04-10 00:38:12.849 UTC [29] LOG:  database system was shut down at 2026-04-10 00:29:49 UTC
db-1 | 2026-04-10 00:38:12.853 UTC [1] LOG:  database system is ready to accept connections
app-1 | Waiting for PostgreSQL...
app-1 | PostgreSQL is up.
app-1 | Running backfill pipeline...
app-1 | =====
app-1 | MASTER PIPELINE
app-1 | =====
app-1 | Running individual pipelines...
app-1 | -----
app-1 | Starting food download...
app-1 | Starting food download step...
app-1 | Found CSV link: https://data.humdata.org/dataset/31579ef5-3895-4002-9ea3-c50857480785/resource/502190c4-0d3d-4084-977e-ef602f053662/download/wfp_food_prices_global_2026.csv
app-1 | Downloaded latest food file to: /app/latest_food_prices.csv
app-1 | -----
app-1 | Starting World Bank pipeline...
app-1 | Starting World Bank Pipeline... mode=backfill, years=2022-2024
app-1 | Range selection: configured backfill range
app-1 | Saved: /app/world_bank_monthly.csv
app-1 | Saved: /app/world_bank_sample10.csv
app-1 | -----
app-1 | [WORLD BANK MONTHLY (processed)] rows=180 cols=7
app-1 | Countries: ['BRA', 'IND', 'NGA', 'PHL', 'USA']
app-1 | Year_Month range: 2022-01 to 2024-12
app-1 | -----
app-1 | Starting food prices ingestion...
app-1 | Starting Food Prices Pipeline...
app-1 | Successfully loaded the master WFP file: wfp_food_prices_master_2023_2025.csv
app-1 | Successfully loaded latest downloaded food file: latest_food_prices.csv
app-1 | Merged historical and latest food data
app-1 | Saved processed file: food_prices.csv
app-1 | Successfully loaded food prices into the database table 'food_prices'!
app-1 | -----
app-1 | Starting energy pipeline...
app-1 | Fetching IEA energy data... mode=backfill, years=2022-2025
app-1 | Saved: energy.csv (180, 8)
app-1 | -----
app-1 | Starting weather backfill pipeline...
app-1 | Starting weather pipeline... mode=backfill, dates=2022-01-01 to 2026-04-09
app-1 | Fetching weather for USA...
app-1 | Fetching weather for BRA...
app-1 | Fetching weather for IND...
app-1 | Fetching weather for PHL...
app-1 | Fetching weather for NGA...
app-1 | Saved: weather.csv (260, 4)
app-1 | Successfully loaded weather data into 'weather'
app-1 | -----
app-1 | All pipelines complete.

```

```

app-1 |
app-1 | Loading to PostgreSQL...
app-1 |
app-1 | =====
app-1 | Dropped existing tables if they existed.
app-1 | Created table 'countries': (5, 2)
app-1 | Creating datedim table...
app-1 | Created table 'datedim' extending to 2036.
app-1 | Created table 'worldbank': (180, 7)
app-1 | Created table 'food_prices': (38, 4)
app-1 | Created table 'energy': (180, 8)
app-1 | Created table 'weather': (260, 4)
app-1 | Created table 'master_data': (260, 17)
app-1 |
app-1 | Adding database primary and foreign keys...
app-1 | =====
app-1 | Added primary key to 'countries'
app-1 | Added primary key and foreign key to 'worldbank'
app-1 | Added primary key and foreign key to 'food_prices'
app-1 | Added primary key and foreign key to 'energy'
app-1 | Added primary key and foreign key to 'weather'
app-1 | Added primary key and foreign key to 'master_data'
app-1 |
app-1 | All primary and foreign keys added successfully.
app-1 | =====
app-1 | All tables created successfully.
app-1 | =====
app-1 | PIPELINE COMPLETE
app-1 | =====
app-1 | Starting cron...
app-1 | Starting Flask API...
app-1 | * Serving Flask app 'app'
app-1 | * Debug mode: on
app-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
app-1 | * Running on all addresses (0.0.0.0)
app-1 | * Running on http://172.18.0.1:8001
app-1 | * Running on http://172.18.0.3:8001
app-1 | Press CTRL+C to quit
app-1 | * Restarting with stat
app-1 | * Debugger is active!
app-1 | * Debugger PIN: 222-862-928
app-1 |
v View in Docker Desktop  o View Config  w Enable Watch  d Detach

```

15.5 Troubleshooting Notes

The following guide addresses common issues encountered during environment setup, deployment, and data retrieval.

Issue	Cause and Resolution
Docker command not found	Docker Desktop is either not installed or not currently running. Ensure Docker is active and fully initialized before executing docker compose commands.
Missing .env file	The project cannot find required environment variables. Verify that .env.example was successfully copied to .env and is saved in the project root.

Invalid/Missing EIA API Key	If the EIA pipeline fails, verify that EIA_API_KEY in the .env file contains a valid key. Re-verify the key through the official EIA registration page if necessary.
Slow Initial Startup	The first execution builds the PostgreSQL warehouse from scratch. Please wait for the initial backfill to complete before assuming a service failure.
API Not Accessible	If localhost:8001 fails to load, run docker compose ps to check container status. If the app container is stopped, review logs via docker compose logs -f app.
Database Connection Errors	Ensure the database container is running and that the DB_credentials in your .env file match the docker-compose.yml configuration.
Empty API Responses	If endpoints return no data, the historical build may still be in progress. Monitor the logs to confirm master.py has finished loading the warehouse tables.
Cron Refresh Failure	If the weekly refresh is missing, verify the cron status inside the container and review /app/cron.log for output from the run_incremental.sh script.
Windows Startup Loop	Symptom: Terminal displays <i>app-1 / PostgreSQL not ready yet. Retrying in 2 seconds...</i> Resolution: This is a known synchronization bug on Windows host machines. To resolve, exit the Docker process entirely, restart Docker Desktop, and re-run: docker compose up --build.

15.6 References

1. [Open-Meteo API](#): Global historical weather and climate data
2. [World Food Programme \(WFP\)](#): Global food price monitoring and commodity data
3. [U.S. Energy Information Administration \(EIA\) API](#): International energy consumption and production statistics
4. [World Bank Open Data API](#): Global macroeconomic indicators and country-level development data

5. [Docker Documentation](#): Containerization standards, Docker Compose orchestration, and persistent volume management
6. [PostgreSQL Documentation](#): Relational database management, SQL syntax, and data integrity constraints
7. [Flask Documentation](#): Python-based micro-framework for RESTful API development and routing